Prasanna Raja. R
2024001019
HPC

Lab - 00

6.

Metric	Bubble Sort	Quick Sort
Execution Time	$O(n^2)$ Slower (≈ 0.24 - 0.45s for 10000 elements)	Faster (≈ 0.11 s for 1,000,000 elements with -O2/-O3
CPU Cycles	–	–
Hotspot Function	bubblesort()	partition()

7.

1. Quick Sort performed significantly better than Bubble Sort:
   * Quick Sort sorted 1,000,000 elements in about 0.11 s with -O2/-O3
   * Bubble Sort required about 0.24 - 0.26 s to sort only 10,000 elements with -O2/-O3.

The difference is due to their time complexities:

* Quick Sort: Average $O(n \log n)$
* Bubble Sort: Average / Worst $O(n^2)$

∴ Quick sort scales much better and is far more efficiently for large datasets.

2. Quick Sort:

The hotspot function was partition(), which consumed 64.29% of the execution time, followed by swap(), ≈ 21.43%.

Bubble Sort:

* At -O0, bubbleSort() consumed 64.29% of the execution time, while swap() consumed 35.71%.

* At -O2 and -O3, the compiler optimized the code so that 100% of the execution time was attributed to bubbleSort().

3. Execution time increases as the input size increases.

* Quick sort grows approximately as $O(n \log n)$, so its execution time increases moderately even for large datasets.

* Bubble Sort grows as $O(n^2)$, so it execution time increases very rapidly as the number of elements. increases.

This is evident because Bubble Sort was tested on only 10,000 elements, whereas Quick Sort efficiently handled 1,000,000 elements.

4. Execution Time

Optimization	Quick Sort	Bubble Sort
-O0	0.34s	0.45s
-O2	0.11s	0.24s
-O3	0.11s	0.26s

* -O2 reduced execution time of Quick sort by approximately 67% compared to -O0, while -O3 provided almost no additional improvement.

* -O2 nearly halved the execution time compared to -O0, in Bubble Sort. In this experiment, -O3 was slightly slower than -O2, showing that higher optimization levels do not always produce better performance.

5. I would recommend Quick Sort for large Dataset.

Justification:

* It has an average time complexity of $O(n \log n)$.

* It sorted 1,000,000 elements in approximately 0.11 seconds with compiler optimization.

* Bubble Sort has $O(n^2)$ complexity and required about 0.24 - 0.26 seconds for only 10,000 elements, indicating that it would become impractically slow for very large datasets.